

pharo-agentic-browser Scripting API

エージェントのオーケストレーションをSmalltalkで

梅澤 真史

<https://github.com/mumez/pharo-agentic-browser>

Scripting API とは？

UI操作なしにSmalltalkコードから AgenticBrowser のトピックをオーケストレーションできるオプションパッケージです。

- 簡単なSmalltalk スクリプトを書くだけで、AgenticBrowser がトピックを作成し、エージェントを実行。結果を受け渡し、すべてが終わるまで待つなどができる
- Spec UI、Web UI に続く3つ目のインターフェース

- **定型的な AI ワークフロー** — 複数ステップからなるワークフローを毎日実行、あるいは CI に組み込む
- **ヘッドレス環境** — ディスプレイがない環境でもトピックを構築・実行
- **複雑なマルチエージェント連携** — UIによる手作業では組みにくい、複雑なエージェントの連携や結果の収集
- **AI によるオーケストレーション記述** — シンプルなDSLにより、コーディングエージェント自身がスクリプトを作成し `st-eval` で実行できる

機能

- 構築に必要なメッセージは `seq:`、`para:`、`topicBy:`、`agentBy:` のわずか数個
- 人間はもちろん、**AI エージェント自身**でも一回の `eval` で書いて実行できるほど平易
- 結果はステップ間で自動的に受け渡される — 手動でのやりとりは不要

```
AgenticBrowser runBy: [ :builder |  
  builder seq: {  
    builder topicBy: [ :t | t prompt: 'List 3 Pharo Smalltalk features.' ].  
    builder topicBy: [ :t | t prompt: 'Write a one-sentence summary of previous features.' ].  
  } agentBy: [ :a | a claude ] ].
```

- 1つのオーケストレーション内のトピックだけでなく、複数のオーケストレーションを連携させる
- 複数のオーケストレーションを `seq:` / `para:` でネスト可能
- グループの中にグループをネスト可能
- **Arena**パターン（同じタスクを異なるエージェントに実行させ、最良の結果を選ぶ）や、複数の調査結果を1つの統合ステップにまとめるといったパターンを実現

- オーケストレーション（およびグループ）は Pharo の Fuel シリアライザで**保存・読み込み**可能
- 実行前に保存 — 構築済みの設定を後で再利用
- 実行後に保存 — 後で読み込んだの確認や、中断していた場合は `resume` が可能

インストール

コアパッケージと合わせて Scripting パッケージをロード:

```
Metacello new  
  baseline: 'AgenticBrowser';  
  repository: 'github://mumez/pharo-agentic-browser:main/src';  
  load: 'Scripting'.
```

テストスイートもロードする場合:

```
Metacello new  
  baseline: 'AgenticBrowser';  
  repository: 'github://mumez/pharo-agentic-browser:main/src';  
  load: 'Scripting-Tests'.
```

スクリプト例

`runBy:` はオーケストレーションを構築してすぐに実行し、完了までブロックします:

```
AgenticBrowser runBy: [ :builder |  
  builder seq: {  
    builder topicBy: [ :t |  
      t title: 'List Pharo features'.  
      t prompt: 'List 3 Pharo Smalltalk features in one sentence each.' ]  
    } agentBy: [ :a | a claude ] ].
```

`scriptBy:` は実行せずに構築だけを行い、後で確認したり `forkRun` したりできます。

各トピックの結果は、次のトピックのプロンプトに引き継がれます:

```
AgenticBrowser runBy: [ :builder |  
  builder seq: {  
    builder topicBy: [ :t |  
      t prompt: 'List 3 Pharo Smalltalk features in one sentence each.' ].  
    builder topicBy: [ :t |  
      t prompt: 'Summarize the feature list from the previous step in one sentence.' ]  
  } agentBy: [ :a | a claude ] ].
```

トピックは並行実行され、結果は次のステップのためにまとめられます:

```
AgenticBrowser runBy: [ :builder |  
  builder para: {  
    builder topicBy: [ :t | t prompt: 'List 3 Pharo Smalltalk language features.' ].  
    builder topicBy: [ :t | t prompt: 'List 3 Pharo Smalltalk development tools.' ]  
  } agentBy: [ :a | a claude ] ].
```

seq: と para: は自由に組み合わせ可能 — 例: 並列調査 → 逐次統合。

異なるエージェントで2つの候補を実行し、勝者を選ぶ:

```
AgenticBrowser groupRunBy: [ :groupBuilder |
  groupBuilder para: {
    groupBuilder orchestrationBy: [ :b |
      b seq: { b topicBy: [ :t | t prompt: 'Implement feature XXX.'. t goal: 'all tests pass' ] }
      agentBy: [ :a | a claude ] ].
    groupBuilder orchestrationBy: [ :b |
      b seq: { b topicBy: [ :t | t prompt: 'Implement feature XXX.'. t goal: 'all tests pass' ] }
      agentBy: [ :a | a codex ] ]
  }.
  groupBuilder singleTopicBy: [ :t | t prompt: 'Pick the best implementation above and open a PR.' ]
  agentBy: [ :a | a kilo ]
].
```

リポジトリのドキュメントに、完全なサンプルがあります:

[to-do-list-orchestration-script.md](#)

- TDD で完全な **Spec2 To-do** リストアプリをゼロから構築
- **6エージェント、7フェーズ**: セットアップ → 並列調査 → 設計 → 実装 → UI テスト → レビュー → ドキュメント作成
- **seq:** / **para:** の双方を使用、フェーズごとに軽量モデルとフルモデルを使い分け、実装ステップは **goal:** でテスト全通過まで駆動
- そのままコピペで使えるスクリプト — 1本のオーケストレーションスクリプトがどこまでできるかを示す実例

AIによるオーケストレーション記述

エージェント自身にオーケストレーションスクリプトを書かせるスキル — 機能を説明するだけで DSL を構築してくれます。

- 自然言語でプロジェクトに欲しい機能を依頼すると、実行可能なオーケストレーションスクリプトへ変換
- 妥当なフェーズにわけたパイプラインを自動的に構成: 計画 (必要な場合) → **TDD 実装** → テスト → リント & スタイルレビュー
- 生成したスクリプトを `docs/scripting-features/feature-<name>.scripting.md` として、着手させる前にプレビューできる

- プレビューではスクリプトを明示的に承認するまで何も実行されない
- 承認後は、対象リポジトリ (`sharedDirectoryPath:`) に対し `st-eval` 経由で実行
- 実行中は、他のオーケストレーションと同様に **Spec UI** や **Web UI** でリアルタイムに進捗を確認可能

ステップが停滞・タイムアウトした場合、スキルは自動的にリトライします。

`AbOrchestrationManager` から手動での確認・リトライも可能です。

このスキルは単なるデモにとどまらず、実際の機能を生み出した

- [RediStick](#) の Time Series サポートは、このスキルが生成したオーケストレーションスクリプトを実行することで実装
- 生成されたスクリプト例: [scripting-features/feature-ts-mrange-mrevrange.scripting.md](#)
- To-Do アプリの例と同じ構成 — 計画/実装/テスト/レビュー — を実際のプロダクションコードベースに適用

実行のバリエーション

同期実行 — `runBy:` / `script run` はすべて完了するまでブロックします。

バックグラウンド実行 — 長時間かかるオーケストレーション向け:

```
script forkRunThen: [ :orc | Transcript crShow: 'Done: ', orc result ].
"... later, if needed:"
script isRunning.
```

`forkRun` は `forkRunThen: [ :orc | ]` の省略形です。

バックグラウンドでオーケストレーションが実行されている間も、**Spec UI** や **Web UI** を開けばリアルタイムに確認できます。thinkingメッセージ、権限確認、トピックの進捗もすべてそこに表示されます。

実行中のオーケストレーションをキャンセル:

```
script terminate.
```

ステップがタイムアウトした後にレジューム（最初の未完了ステップから、記録済みの結果を再利用して再開）:

```
script resume.
```

保存と読み込み

Fuel でオーケストレーション（やグループ）を永続化（実行前/後どちらも可）：

```
script save.                                     "-> <sharedDirectoryPath>/<name>.fuel"
script saveTo: '/path/to/my-script.fuel' asFileReference.

loaded := AbTopicOrchestration loadFrom: '/path/to/my-script.fuel'.
loaded result.                                   "results from the saved run"
loaded resume.                                  "continue if the saved run stopped early"
```

保存したオーケストレーションは `orchestrationLoadFrom:` を使って新しいグループにそのまま読み込むこともできます。

設定

設定	デフォルト	適用範囲
<code>orchestrationStepWaitTimeoutSeconds</code>	900秒	トピックのステップごと
<code>orchestrationGroupItemWaitTimeoutSeconds</code>	3600秒	<code>para:</code> 内で実行されるグループ項目ごと

グローバル、またはオーケストレーション/グループごとに調整可能:

```
script settings orchestrationStepWaitTimeoutSeconds: 1800.
```

設定	デフォルト	効果
<code>lingerOrchestrationTopicsAfterRun</code>	<code>false</code>	オーケストレーション完了後もトピックを Topic Manager に残すかどうか

- `false` — 実行完了後、トピックは Topic Manager から削除される
- `true` — トピックが残り、後から UI で実行全体の進捗をレビュー可能

```
script settings lingerOrchestrationTopicsAfterRun: true.
```

Spec UI や Web UI で、結果に至るまでの経緯を後で振り返りたい場合は `true` に設定します。

まとめ

Scripting API は、コード駆動によるヘッドレスなオーケストレーションを AgenticBrowser にもたらしめます:

- シンプルな DSL — `seq:` / `para:` / `topicBy:` / `agentBy:`、人間にも AI エージェントにも扱いやすい
- オーケストレーショングループ — ワークフロー全体を並列・逐次・ネストで構成
- 結果のルーティング — ステップ間の受け渡しは自動。手動の操作は不要
- 柔軟な実行 — 同期/バックグラウンド実行、キャンセル/レジュームに対応
- Fuel による永続化 — オーケストレーションとその結果を保存・再読み込み

フィードバックとコントリビューションをお待ちしています！

<https://github.com/mumez/pharo-agentic-browser>